

Next.js Development Course Syllabus

Archer Infotech, Kothrud Pune | Classroom & live-online batches | Updated September 2026

Course Overview

This Next.js course is designed to take learners from React fundamentals to building production-ready, full-stack web applications using the modern **Next.js App Router** architecture.

The syllabus focuses on current Next.js 16.x practices, including React Server Components, Client Components, Server Functions/Server Actions, Route Handlers, Cache Components, streaming, Suspense, partial rendering, authentication, databases, SEO, testing, security, performance, deployment, and AI-assisted development.

The course also introduces the older **Pages Router** so learners can maintain existing enterprise applications while following the App Router for new projects.

1. Prerequisites

Before learning Next.js, students should understand:

HTML

- HTML document structure
- Semantic HTML
- Forms
- Inputs
- Tables
- Lists
- Links
- Images
- Accessibility basics

CSS

- Selectors
- Box model
- Positioning
- Flexbox
- CSS Grid
- Responsive design
- Media queries
- CSS variables
- Transitions and animations

JavaScript

- Variables
- Data types
- Operators
- Functions
- Arrays
- Objects
- Classes
- Modules
- Promises
- Async/await
- Error handling
- ES6+ features

React

- JSX
- Functional components
- Props
- State
- Event handling
- Forms
- Hooks
- useState
- useEffect
- useContext
- useRef
- Component composition

2. Modern JavaScript for Next.js**ES6+ Features**

- let and const
- Arrow functions
- Template literals
- Object shorthand
- Destructuring
- Spread syntax
- Rest parameters
- Optional chaining
- Nullish coalescing

Modules

- Named exports
- Default exports
- Dynamic imports

Asynchronous JavaScript

- Promises
- Async/await
- Parallel requests
- Promise.all
- Error handling

Array Methods

- map
- filter
- reduce
- find
- some
- every

3. TypeScript for Next.js**TypeScript Fundamentals**

- Primitive types
- Arrays
- Tuples

- Objects
- Interfaces
- Type aliases
- Union types
- Intersection types

Functions

- Parameter types
- Return types
- Optional parameters
- Generics

React and Next.js Types

- Component props
- Children
- Event types
- Route parameters
- API models

Utility Types

- Partial
- Required
- Pick
- Omit
- Record

Type Safety Best Practices

- Avoiding unnecessary any
- Narrowing
- Runtime validation vs compile-time types

4. Introduction to Next.js

What is Next.js?

- React framework for web applications
- Full-stack React development
- Server and client rendering
- File-system routing

Why Next.js?

- Routing
- Server rendering
- Static rendering
- API/backend capabilities
- Performance optimization
- SEO
- Image optimization
- Font optimization
- Production tooling

Next.js vs React

- Library vs framework
- Routing
- Data fetching

- Backend capabilities
- Rendering strategies

Next.js Use Cases

- SaaS applications
- E-commerce
- Marketing websites
- Blogs
- Dashboards
- Portals
- Full-stack applications

5. Current Next.js Architecture

App Router

- Modern Next.js routing system
- app directory
- Server Components
- Suspense
- Server Functions

Rendering Model

- Server-first architecture
- Client interactivity where required

Modern Next.js Principles

- Server Components by default
- Minimize Client Components
- Fetch data close to where it is used
- Cache intentionally
- Stream slow UI
- Keep security checks on the server

6. Development Environment

Required Tools

- Node.js
- npm / pnpm / yarn
- VS Code
- Git
- GitHub
- Browser developer tools

Creating a Project

- create-next-app
- TypeScript setup
- ESLint
- Tailwind CSS option
- App Router
- src directory
- Import aliases

Running the Application

- Development server

- Production build
- Production server

7. Next.js Project Structure

Important Files

- package.json
- next.config.ts
- tsconfig.json
- eslint.config.*
- .env.local

App Directory

- app
- layout.tsx
- page.tsx
- loading.tsx
- error.tsx
- not-found.tsx
- route.ts

Supporting Directories

- components
- lib
- services
- actions
- types
- utils
- public

8. File-System Routing

Pages

- Creating routes with folders
- page.tsx

Route Segments

- Static routes
- Nested routes

Dynamic Routes

- [id]
- [slug]

Catch-All Routes

- [...slug]

Optional Catch-All Routes

- [...slug?]

Route Parameters

- Async route params
- Typed parameters

9. Layouts

Root Layout

- Application shell
- HTML and body
- Global styles

Nested Layouts

- Shared navigation
- Dashboards
- Section layouts

Layout Persistence

- Preserving UI between navigation

Layout Best Practices

- Keep shared UI in layouts
- Avoid unnecessary client boundaries

10. Templates

Layout vs Template

- Persistent layout behavior
- Remounting templates

Template Use Cases

- Page transitions
- Per-navigation effects
- Resetting local component state

11. Navigation

Link

- Client-side navigation
- Prefetching

Programmatic Navigation

- Router navigation
- Redirects

Active Navigation

- Current pathname
- Selected navigation states

Navigation Performance

- Prefetching
- Streaming
- Cached route shells

12. Route Groups

Route Groups

- (group) folders

Use Cases

- Organizing routes
- Multiple layouts
- Avoiding URL changes

Examples

- (marketing)
- (dashboard)
- (auth)

13. Parallel Routes

Parallel Route Concepts

- Multiple slots
- Independent route areas

@slot

- Slot conventions

Use Cases

- Dashboards
- Split views
- Conditional layouts

14. Intercepting Routes

Intercepting Route Concepts

- Showing routes within the current context

Common Use Cases

- Modals
- Photo previews
- Product quick views

Navigation Behavior

- Soft navigation
- Direct URL navigation

15. Server Components

React Server Components

- Server Components by default
- Server-side rendering of component trees

Benefits

- Direct database access
- Reduced client JavaScript
- Secure server-side logic
- Server-side data fetching

Server Component Use Cases

- Database queries
- CMS queries
- Static content
- Secure business logic

Server Component Limitations

- No browser APIs
- No client event handlers
- No stateful client hooks

16. Client Components

'use client'

- Defining a client boundary

Use Client Components For

- State
- Event handlers
- Browser APIs
- `useEffect`
- Interactive widgets

Client Component Design

- Push client boundaries downward
- Avoid marking entire layouts as client components

Passing Data

- Serializable props
- Server-to-client composition

17. Server and Client Component Composition

Composition Patterns

- Server parent + client child
- Client wrapper + server children
- Shared UI patterns

Data Boundaries

- What can cross the server/client boundary

Security

- Preventing secrets from reaching client bundles

Performance

- Reducing JavaScript shipped to browsers

18. Data Fetching in Server Components

Async Server Components

- Fetching directly inside components

Data Sources

- REST APIs
- GraphQL
- Databases
- CMS systems
- Internal services

Parallel Fetching

- Avoiding waterfalls

- `Promise.all`

Sequential Fetching

- Dependent requests

Data Fetching Best Practices

- Fetch where data is consumed
- Avoid unnecessary client fetching

19. Loading UI and Streaming

`loading.tsx`

- Route-level loading UI

Suspense

- Streaming component sections

Streaming

- Sending UI progressively
- Faster perceived performance

Loading Boundaries

- Skeletons
- Placeholders
- Independent async sections

20. Error Handling

`error.tsx`

- Route-level errors
- Recovery UI

Global Errors

- Global fallback concepts

Expected vs Unexpected Errors

- Validation failures
- Database failures
- Network failures

Server Function Errors

- Returning safe error states

Security

- Avoid exposing internal server details

21. Not Found Handling

`notFound()`

- Triggering 404 behavior

`not-found.tsx`

- Custom 404 experiences

Dynamic Resource Handling

- Missing blog posts
- Missing products
- Missing users

22. Redirects

`redirect()`

- Server-side redirects

Permanent Redirects

Redirect Configuration

Common Use Cases

- Authentication
- Legacy URLs
- Resource changes
- Post-mutation navigation

23. Route Handlers

`route.ts`

- Building HTTP endpoints

Supported HTTP Methods

- GET
- POST
- PUT
- PATCH
- DELETE
- HEAD
- OPTIONS

Request and Response

- Web Request API
- Web Response API

Use Cases

- REST APIs
- Webhooks
- File responses
- JSON APIs
- Backend-for-frontend endpoints

24. Route Handler Input

URL Parameters

- Dynamic route context

Query Parameters

- Search
- Filters
- Pagination

Headers

Cookies

Request Body

- JSON
- FormData
- Text
- Binary data

25. Backend-for-Frontend Architecture

Next.js as a Backend Layer

- UI-specific APIs
- Aggregating upstream APIs
- Hiding backend complexity

Good Use Cases

- Authentication callbacks
- Secure proxying
- Webhooks
- UI-specific data aggregation

Limitations

- Long-running workloads
- Queue workers
- Specialized backend services

26. Server Functions and Server Actions

Server Functions

- Async functions that execute on the server

'use server'

- File-level usage
- Function-level usage

Server Actions

- Server Functions used for mutations/actions

Calling Server Actions

- Forms
- Buttons
- Client Components

Benefits

- Server-side mutation logic
- Reduced API boilerplate
- UI + data updates in one workflow

27. Forms with Server Actions

Native Form Actions

- `action`
- `formAction`

FormData

- Reading form fields

Validation

- Server-side validation
- Client-side validation

Submission States

- Pending state
- Success state
- Error state

Progressive Enhancement

- Forms that work without complex client JavaScript

28. React Action Hooks

useActionState

- Mutation state
- Validation responses

useFormStatus

- Pending state
- Submit button feedback

Optimistic UI

- useOptimistic

Use Cases

- Comments
- Likes
- Cart updates
- Inline editing

29. Data Validation

Why Runtime Validation Is Needed

- TypeScript is compile-time only

Schema Validation

- Zod
- Valibot concepts
- Other schema libraries

Validating

- Forms
- Route Handler input
- Search params
- Environment variables

Error Mapping

- Field-level messages
- Form-level errors

30. Database Integration

Connecting Databases

- PostgreSQL
- MySQL
- MongoDB
- SQLite

ORM Options

- Prisma
- Drizzle
- Other database clients

Server-Only Access

- Database connections from server code

CRUD

- Create
- Read
- Update
- Delete

Data Modeling

- Relations
- Indexes
- Constraints

31. Prisma with Next.js

Prisma Fundamentals

- Schema
- Models
- Client

Migrations

CRUD Operations

Relations

Seeding

Production Considerations

- Connection management
- Serverless database patterns

32. Authentication Fundamentals

Authentication Concepts

- Sign up
- Sign in
- Sign out
- Sessions

Authentication Approaches

- Session-based
- Token-based
- OAuth/OIDC

Authentication Libraries

- Auth.js concepts
- Third-party identity providers

Enterprise Authentication

- Microsoft Entra ID concepts
- Google
- GitHub
- Corporate SSO

33. Authorization**Authentication vs Authorization****Role-Based Access Control**

- User
- Manager
- Admin

Permission-Based Authorization**Server-Side Enforcement**

- Server Components
- Server Functions
- Route Handlers

UI-Level Permissions

- Conditional display

Security Rule

- Never rely only on hidden UI

34. Sessions and Cookies**Cookies**

- Reading cookies
- Setting cookies

Secure Cookie Attributes

- HTTP-only
- Secure
- SameSite

Session Storage**User Context**

- Server-side session lookup

35. Middleware / Proxy Layer Concepts**Request-Time Logic**

- Authentication redirects
- URL rewriting
- Localization
- Request headers

Appropriate Use

- Lightweight request logic

Avoid

- Heavy database work
- Expensive business logic

Runtime Considerations

- Node/runtime capabilities
- Deployment platform behavior

36. Caching Fundamentals

Why Cache?

- Reduce repeated work
- Improve response time
- Reduce database/API load

Cache Scope

- Pages
- Components
- Functions
- Data

Static vs Dynamic Data**Cache Invalidation**

- Time-based
- Event-based

37. Cache Components

Cache Components Model

- Modern caching architecture

Enabling Cache Components

- `cacheComponents` configuration

'use cache'

- Function-level cache
- Component-level cache
- File-level cache

Cached and Dynamic UI

- Combining cached and request-time content

Partial Prerendering Concepts

- Static shell
- Dynamic streamed content

38. Cache Lifetime

`cacheLife`

- Defining freshness

Cache Profiles

- Reusable cache policies

Choosing Cache Duration

- Product catalog
- News content
- User-specific data

Staleness Tradeoffs

- Performance vs freshness

39. Cache Tags and Revalidation**Cache Tagging****revalidateTag**

- Revalidating tagged data

updateTag

- Immediate invalidation/update workflows

revalidatePath

- Route/path invalidation

Revalidation Strategy

- Mutation-driven freshness
- CMS webhooks

40. Private and Remote Caching Concepts**Private Caching**

- User-specific cached data

Remote Cache

- Platform-provided cache handlers

Compliance Considerations**Choosing Cache Scope Carefully****41. Request-Time Data****Cookies****Headers****Search Parameters****Authentication State****Dynamic Rendering**

- Request-dependent UI

Cache Boundary Rules

- Passing runtime data into cached functions where appropriate

42. Static Rendering

Static Content

- Build-time rendering concepts

Use Cases

- Marketing pages
- Documentation
- Blog content

Benefits

- Speed
- CDN caching
- SEO

43. Dynamic Rendering**Request-Time Rendering**

- Personalized pages
- Authenticated dashboards
- Real-time data

Dynamic Data Sources

- Cookies
- Headers
- User sessions

Performance Considerations

- Streaming
- Selective caching

44. Incremental and Partial Rendering Concepts**Incremental Freshness**

- Revalidating cached content

Partial Prerendering

- Static route shell
- Streamed dynamic sections

Benefits

- Fast navigation
- Personalized content
- Reduced full-page blocking

45. Instant Navigation Concepts**Navigation Responsiveness**

- Fast route transitions

Route Shells**Partial Prefetching Concepts****Streaming and Cache Cooperation****Designing App-Like Experiences**

- Immediate feedback

- Optimistic UI
- Preserved client state

46. Metadata API

Static Metadata

- Title
- Description

Dynamic Metadata

- generateMetadata

Metadata Fields

- Open Graph
- Twitter cards
- Robots
- Canonical URLs

Dynamic Product/Post Metadata

47. SEO

SEO Fundamentals

- Crawlability
- Indexability
- Search intent

Technical SEO

- Metadata
- Canonical URLs
- Sitemaps
- Robots.txt

Structured Data

- JSON-LD
- Product schema
- Article schema
- Organization schema

Rendering and SEO

- Server-rendered content
- Static content

48. Sitemap and Robots

Sitemap Generation

- Static sitemap
- Dynamic sitemap

Robots Configuration

- Indexing rules

Large Websites

- Multiple sitemap concepts

49. Open Graph Images

Social Preview Images

Dynamic OG Image Generation

Use Cases

- Blog posts
- Products
- User profiles
- Course pages

50. Image Optimization

next/image

- Responsive images
- Optimization

Image Sizing

- Width and height
- Fill

Remote Images

Priority Loading

Performance

- Avoiding layout shift
- Efficient formats

51. Font Optimization

next/font

Local Fonts

Google Fonts

Performance Benefits

- Reduced layout shift
- Self-hosting

Typography Strategy

52. Styling Next.js Applications

Global CSS

CSS Modules

Tailwind CSS

Sass

CSS-in-JS Considerations

UI Libraries

- shadcn/ui
- Material UI concepts

- Chakra UI concepts

Design Systems

- Tokens
- Reusable components

53. Responsive Design

Mobile-First Development

Breakpoints

Responsive Navigation

Responsive Grids

Responsive Images

Accessible Mobile UI

54. State Management

State Categories

- Server state
- URL state
- Local client state
- Shared client state

Local React State

- useState
- useReducer

Context API

Zustand

Redux Toolkit

Principle

- Do not move server data into client state unnecessarily

55. URL State

Search Parameters

- Search
- Filtering
- Sorting
- Pagination

Benefits

- Shareable URLs
- Back/forward navigation
- Server-renderable state

URL-Driven Applications

- Product filters
- Admin tables

56. Client Data Fetching

When Client Fetching Is Appropriate

- Frequently updating client-only data
- Browser-only APIs

Common Libraries

- SWR
- TanStack Query

Client Fetching Patterns

- Loading
- Error
- Refetch
- Mutation

Server vs Client Fetching

- Choosing intentionally

57. Suspense

Suspense Fundamentals

Loading Boundaries

Streaming Server Components

Nested Suspense

Avoiding Full-Page Blocking

58. React Compiler and Rendering Optimization

React Compiler Concepts

- Automatic memoization

Performance Mindset

- Avoid premature manual memoization

Component Boundaries

Client JavaScript Reduction

Profiling Before Optimization

59. Code Splitting

Route-Based Code Splitting

Dynamic Imports

Lazy Components

Heavy Client Libraries

Bundle Reduction

60. Turbopack

Turbopack Fundamentals

- Next.js bundler

Development Builds**Production Builds****Incremental Compilation****Persistent Caching Concepts****Debugging Build Issues****61. Environment Variables**

`.env`

`.env.local`

Server Variables**Public Variables**

- `NEXT_PUBLIC_*`

Environment Separation

- Development
- Staging
- Production

Security

- Never expose secrets to client code

62. Configuration

`next.config.ts`

Common Configuration

- Images
- Redirects
- Rewrites
- Headers
- Cache Components
- Build configuration

Type-Safe Configuration**63. Redirects and Rewrites****Redirects**

- Permanent
- Temporary

Rewrites

- Proxy-style URL mapping

Use Cases

- Legacy migrations
- Backend proxies

- Clean public URLs

64. HTTP Headers

Security Headers

Cache Headers

Content-Type

Custom Headers

Platform/CDN Considerations

65. Security Fundamentals

Server/Client Boundary Security

- Keep secrets server-side

Authentication Checks

Authorization Checks

Input Validation

Output Encoding

Injection Prevention

Dependency Security

66. Data Security

Protecting Server Functions

- Authenticate every sensitive action
- Authorize mutations

Route Handler Security

Preventing Sensitive Data Exposure

Data Access Layer

- Centralized authorization patterns

Least Privilege

67. XSS, CSRF, and Injection Concepts

Cross-Site Scripting

Dangerous HTML Rendering

CSRF Considerations

SQL Injection

- Parameterized queries
- ORM safety

Secure Form Processing

68. Content Security Policy

CSP Fundamentals

Script Restrictions

Nonces

Third-Party Scripts

Reporting

Production Security Strategy

69. File Uploads

File Input Handling

FormData

Route Handler Uploads

Object Storage Concepts

- S3
- Azure Blob Storage
- Cloud storage

Validation

- File type
- File size

Security

- Untrusted files

70. Email Integration

Transactional Email

- Registration
- Password reset
- Notifications

Email Providers

- Provider-agnostic concepts

Server-Only Sending

Email Templates

Error Handling

71. Payments Integration

Payment Flow Concepts

Checkout Session

Webhooks

Server-Side Verification

Order State

Idempotency

Security Considerations

72. Webhooks

Webhook Endpoints

- Route Handlers

Signature Verification

Idempotency

Retry Handling

Common Webhooks

- Payments
- CMS
- Authentication
- Git providers

73. Real-Time Features

WebSocket Concepts

Server-Sent Events

Hosted Realtime Services

Use Cases

- Chat
- Notifications
- Live dashboards
- Collaboration

Deployment Considerations

74. Internationalization

Locale Routing

Translation Dictionaries

Server-Side Locale Loading

Date and Currency Formatting

SEO for Multiple Languages

75. Accessibility

Semantic HTML

Keyboard Navigation

Focus Management

ARIA

Accessible Forms

Accessible Navigation

Automated and Manual Testing

76. Testing Fundamentals

Testing Pyramid

- Unit tests
- Integration tests
- E2E tests

Testable Architecture

- Separate business logic
- Mock external services

What to Test

- Components
- Server Functions
- Route Handlers
- User flows

77. Unit Testing

Vitest / Jest Concepts

Testing Utilities

Pure Functions

Validation Schemas

Data Access Logic

Mocking Dependencies

78. Component Testing

React Testing Library

Rendering Client Components

User Events

Forms

Validation States

Accessibility-Focused Queries

79. Server Component Testing Concepts

Testing Server Logic

Separating Data Functions

Integration-Oriented Testing

Mocking Databases/APIs Carefully

Avoiding Fragile Implementation Tests

80. Route Handler Testing

Testing HTTP Methods

Request Validation

Response Status

Authentication

Authorization

Error Handling

81. End-to-End Testing

Playwright

Cypress Concepts

Critical User Flows

- Registration
- Login
- Checkout
- CRUD
- Search

Test Environments

Seed/Test Data

82. Performance Optimization

Core Web Vitals

Reduce Client JavaScript

Optimize Images

Optimize Fonts

Stream Slow UI

Cache Expensive Work

Avoid Request Waterfalls

Use Dynamic Imports

83. Observability

Application Logging

Error Tracking

Performance Monitoring

Web Vitals

Request Tracing Concepts

Production Debugging

84. Analytics

Product Analytics

Web Analytics

Conversion Tracking

Privacy Considerations

Server vs Client Events

85. Deployment Fundamentals

Production Build

Environment Variables

Runtime Requirements

Static Assets

Cache Behavior

Domain Configuration

86. Vercel Deployment

Connecting Git Repository

Preview Deployments

Production Deployment

Environment Variables

Domains

Logs

Analytics and Performance Features

87. Self-Hosting

Node.js Server

Docker

Reverse Proxy

- Nginx concepts

CDN

Cache Coordination

Production Process Management

Scaling Considerations

88. Cloud Deployment

AWS Concepts

- Containers
- Serverless
- CDN

Azure Concepts

- App Service
- Containers
- Static/web hosting patterns

Google Cloud Concepts

OpenNext / Adapter Ecosystem Concepts

Platform Differences

89. Docker for Next.js

Docker Fundamentals

Multi-Stage Builds

Standalone Output Concepts

Environment Variables

Container Security

Production Image Optimization

90. CI/CD

Continuous Integration

- Install
- Lint
- Type check
- Test
- Build

Preview Environments

Production Deployment

GitHub Actions

Azure DevOps

GitLab CI

Deployment Gates

91. Git and GitHub

Git Fundamentals

- Clone
- Add
- Commit

- Push
- Pull

Branching Strategy**Pull Requests****Code Reviews****Merge Conflicts****Release Tags****Professional GitHub Portfolio****92. Code Quality****ESLint****Prettier****TypeScript Strict Mode****Naming Conventions****Component Design****Separation of Concerns****Avoiding Duplicate Code****Server/Client Boundary Review****93. Application Architecture****Feature-Based Architecture****Suggested Layers**

- Routes/pages
- UI components
- Server actions
- Data access
- Domain logic
- Validation
- Authentication

Shared Libraries**Avoiding God Components****Keeping Business Logic Out of Templates****94. Data Access Layer****Centralized Database Access****Query Functions****Authorization-Aware Data Access****Validation**

Reusability**Testing****Preventing Accidental Client Imports****95. Repository Patterns****Single Application Repository****Monorepo Concepts****Shared Packages****Turborepo Concepts****Common Packages**

- UI
- Config
- Types
- Utilities

Enterprise Team Workflows**96. Pages Router – Legacy and Maintenance**

The App Router should be the primary learning path for new projects, but students should understand the Pages Router for existing applications.

Pages Router Fundamentals

- pages
- _app
- _document

Legacy Routing

- Dynamic pages

Data Fetching

- getStaticProps
- getServerSideProps
- getStaticPaths

API Routes**Migration to App Router**

- Incremental migration concepts

97. Migrating Existing Next.js Applications**Pages Router to App Router****Shared Component Reuse****Server vs Client Boundaries****Data Fetching Migration****API Routes to Route Handlers****Legacy Caching to Cache Components**

Testing Migration Strategy

98. Next.js Development with AI Assistance

AI-Assisted Development

- Generate components
- Generate route handlers
- Refactor Server/Client boundaries
- Generate tests
- Debug build errors
- Explain rendering behavior

Version-Aware AI Usage

- Give the exact Next.js version
- Use version-matched documentation
- Avoid outdated Pages Router answers

AI Code Review

- Security
- Caching
- Server/client boundary
- Type safety
- Performance

Responsible Usage

- Validate generated code
- Review package APIs
- Protect confidential information

99. AI-Ready Development Workflow

Repository Guidance

- Project architecture instructions
- Coding conventions
- Version-specific rules

Documentation-Aware Agents

- Current framework docs
- Project documentation
- API contracts

Agent Tasks

- Feature implementation
- Test generation
- Refactoring
- Migration
- Performance optimization

Human Review

- Security validation
- Architectural review
- Business logic validation

100. Mini Project 1 – Blog Application

Features

- App Router
- Dynamic routes
- Server Components
- Markdown/MDX or CMS data
- Metadata
- SEO
- Sitemap

Concepts

- Static content
- Dynamic metadata
- Route params
- Server rendering

101. Mini Project 2 – Product Catalog**Features**

- Product listing
- Product details
- Search
- Filters
- Sorting
- Pagination

Concepts

- Search params
- Server Components
- Caching
- Dynamic routes

102. Mini Project 3 – Authentication Portal**Features**

- Registration
- Login
- Logout
- Protected dashboard
- Profile

Concepts

- Authentication
- Sessions
- Cookies
- Authorization
- Server-side security

103. Mini Project 4 – Task Management Application**Features**

- Create task
- Edit task
- Delete task

- Status updates
- Filters

Concepts

- Server Actions
- Forms
- Validation
- Optimistic updates
- Database CRUD

104. Mini Project 5 – Admin Dashboard**Features**

- Authentication
- Tables
- Search
- Pagination
- Charts
- Role-based access

Concepts

- Layouts
- Parallel data fetching
- Server Components
- Client chart components

105. Mini Project 6 – E-Commerce Cart**Features**

- Products
- Cart
- Quantity update
- Checkout flow
- Orders

Concepts

- Server Actions
- Client state
- Database
- Payments concepts
- Revalidation

106. Mini Project 7 – SaaS Application**Features**

- Multi-user authentication
- Dashboard
- CRUD
- Subscription concepts
- Settings
- Audit activity

Concepts

- Full-stack architecture

- Authorization
- Database
- Secure Server Functions
- Deployment

107. Major Capstone Project – E-Commerce Platform

Customer Features

- Registration
- Login
- Product catalog
- Search
- Filters
- Cart
- Checkout
- Orders
- Profile

Admin Features

- Product management
- Inventory
- Orders
- Customers
- Dashboard

Technical Requirements

- App Router
- Server Components
- Client Components
- Server Actions
- Route Handlers
- Database
- Authentication
- Authorization
- Caching
- Revalidation
- Testing
- Deployment

108. Major Capstone Project – Learning Management System

Features

- Student login
- Instructor login
- Course catalog
- Enrollment
- Lessons
- Progress
- Quizzes
- Admin dashboard

Technical Areas

- Role-based permissions
- Nested routes

- Data access layer
- Forms
- Server Actions
- Media/file handling
- SEO for public courses

109. Major Capstone Project – Project Management SaaS

Features

- Organizations
- Users
- Projects
- Tasks
- Kanban board
- Comments
- Notifications
- Activity log

Advanced Concepts

- Multi-tenant authorization
- Optimistic UI
- Real-time concepts
- Audit logs
- Production architecture

110. Capstone Project Requirements

Every capstone should demonstrate:

- Next.js App Router
- TypeScript
- React Server Components
- Client Components
- Nested layouts
- Dynamic routes
- Loading UI
- Error handling
- Server Functions/Server Actions
- Route Handlers
- Form validation
- Database integration
- Authentication
- Authorization
- Cache strategy
- Revalidation
- Metadata and SEO
- Responsive UI
- Accessibility
- Testing
- Git/GitHub
- Deployment
- Documentation

111. Interview Preparation – Next.js

Core Questions

- Next.js vs React
- App Router vs Pages Router
- Server vs Client Components
- 'use client'
- 'use server'
- Server Functions
- Server Actions
- Route Handlers
- Layouts
- Dynamic routes
- Streaming
- Suspense
- Caching
- Cache Components
- Revalidation
- Static vs dynamic rendering
- Authentication
- Middleware/proxy concepts
- Metadata
- Image optimization

112. Interview Preparation – Architecture

Architecture Questions

- How would you structure a large Next.js application?
- Where should database access live?
- When should a component be client-side?
- How do you prevent data leaks?
- How do you cache a product catalog?
- How do you invalidate cache after mutation?
- When should you use a Route Handler vs Server Action?
- How would you implement authentication?
- How would you deploy globally?

113. Practical Interview Exercises

Candidates should be able to:

- Create dynamic routes
- Build nested layouts
- Fetch server-side data
- Build a Client Component
- Write a Server Action
- Validate form input
- Write a Route Handler
- Implement authentication checks
- Protect an admin page
- Add metadata
- Optimize an image
- Implement caching
- Revalidate data after mutation
- Write a Playwright test
- Explain a server/client boundary

114. Recommended Modern Next.js Stack

A modern training stack may include:

- Next.js 16.x / 16.3.x release line
- React 19.x
- TypeScript
- App Router
- React Server Components
- Server Functions / Server Actions
- Route Handlers
- Cache Components
- Zod
- PostgreSQL
- Prisma or Drizzle
- Auth.js or enterprise identity provider
- Tailwind CSS
- shadcn/ui or another component system
- Vitest/Jest
- React Testing Library
- Playwright
- ESLint
- Git
- GitHub
- Vercel / Azure / AWS deployment

115. Suggested Course Sequence

Phase 1 – Foundations

- HTML/CSS refresher
- Modern JavaScript
- TypeScript
- React fundamentals

Phase 2 – Next.js Fundamentals

- Next.js introduction
- Project setup
- App Router
- Pages and layouts
- Dynamic routes
- Navigation
- Server Components
- Client Components

Phase 3 – Full-Stack Development

- Data fetching
- Route Handlers
- Server Functions
- Server Actions
- Forms
- Validation
- Database integration

Phase 4 – Production Application Features

- Authentication

- Authorization
- Sessions
- File uploads
- Email
- Webhooks
- Payments concepts

Phase 5 – Rendering and Performance

- Streaming
- Suspense
- Cache Components
- Cache lifetime
- Revalidation
- Partial rendering
- Performance optimization

Phase 6 – SEO and Quality

- Metadata
- SEO
- Accessibility
- Security
- Testing
- Observability

Phase 7 – Deployment

- Vercel
- Docker
- Self-hosting
- Cloud deployment
- CI/CD

Phase 8 – Professional Development

- Git and GitHub
- Architecture
- Code quality
- AI-assisted development

Phase 9 – Career Preparation

- Mini projects
- Capstone project
- Portfolio
- Interview preparation
- Mock interviews

116. Learning Outcomes

After completing this course, learners should be able to:

- Build modern full-stack applications with Next.js
- Use the App Router confidently
- Design Server and Client Component boundaries
- Build nested and dynamic routes
- Fetch data efficiently
- Stream slow UI
- Build REST-style Route Handlers

- Use Server Functions and Server Actions
- Validate user input
- Integrate relational or NoSQL databases
- Implement authentication and authorization
- Build secure session-based applications
- Use Cache Components
- Design caching and revalidation strategies
- Optimize images, fonts, and bundles
- Implement metadata and technical SEO
- Build accessible user interfaces
- Write unit, integration, and E2E tests
- Deploy to managed and self-hosted environments
- Use Git/GitHub professionally
- Build portfolio-ready full-stack projects
- Maintain older Pages Router applications
- Prepare for Next.js interviews

117. Suggested Course Deliverables

By the end of the course, each student should ideally have:

- TypeScript exercises
- React revision exercises
- Routing assignments
- Server Component exercises
- Server Action exercises
- API/Route Handler exercises
- Database assignments
- Authentication assignment
- Caching assignment
- Testing exercises
- 5–7 mini projects
- At least one major capstone project
- GitHub repositories
- Professional README files
- Deployed applications
- Interview preparation
- Mock interviews
- AI-assisted development practice
- A full-stack web development portfolio

118. Optional Advanced Modules

Advanced Cache Components

- Complex cache boundaries
- Private cache
- Remote cache
- Custom cache strategies

Multi-Tenant SaaS

- Organization isolation
- Tenant-aware data access
- Authorization

Micro Frontends

- Composition concepts
- Independent deployment tradeoffs

Monorepos

- Turborepo
- Shared packages
- Design systems

GraphQL

- GraphQL APIs
- Server-side clients
- Typed schemas

Real-Time Collaboration

- Presence
- Live cursors
- Collaborative editing concepts

AI Applications with Next.js

- LLM chat interfaces
- Streaming responses
- Retrieval-based applications
- AI tool calling concepts
- Secure server-side model access

119. Career Roles After Next.js Training

This course can help learners prepare for roles such as:

- Next.js Developer
- React Developer
- Frontend Developer
- Full-Stack JavaScript Developer
- Full-Stack TypeScript Developer
- Web Application Developer
- Frontend Software Engineer
- Full-Stack Software Engineer
- SaaS Application Developer

Conclusion

Next.js has evolved into a full-stack React framework that combines server rendering, client interactivity, backend endpoints, server-side mutations, caching, streaming, SEO, and production deployment in one framework.

A production-ready Next.js developer should understand much more than routing and React components. They should know how to design Server and Client Component boundaries, fetch and mutate data securely, integrate databases, build authentication and authorization, use modern caching and revalidation, optimize performance, test applications, and deploy them reliably.

This syllabus provides a structured path from React fundamentals to modern Next.js full-stack application development and professional interview preparation.

Enquire or book a free demo session. Archer Infotech, Flat No. 12, Divyadarshan Housing Society, behind Kothrud Bus Stand Road, Kothrud, Pune 411038. Phone +91 9850 678451. Email info@archerinfotech.in. Web archerinfotech.in.